

TripletMarginWithDistanceLoss#

<https://pytorch.org/docs/stable/generated/torch.nn.modules.loss.TripletMarginWithDistanceLoss.html>

Description:

Creates a criterion that measures the triplet loss given input tensors `aaa`, `ppp`, and `nnn` (representing anchor, positive, and negative examples, respectively), and a nonnegative, real-valued function (“distance function”) used to compute the relationship between the anchor and positive example (“positive distance”) and the anchor and negative example (“negative distance”). The unreduced loss (i.e., with reduction set to 'none') can be described as: where `NNN` is the batch size; `ddd` is a nonnegative, real-valued function quantifying the closeness of two tensors, referred to as the `distance_function`; and `marginmarginmargin` is a nonnegative margin representing the minimum difference between the positive and negative distances that is required for the loss to be 0. The input tensors have `NNN` elements each and can be of any shape that the distance function can handle. If reduction is not 'none' (default 'mean'), then: See also `TripletMarginLoss`, which computes the triplet loss for input tensors using the `lpl_plp` distance as the distance function.

Function Signature

```
class torch.nn.modules.loss.TripletMarginWithDistanceLoss(*,  
distance_function=None, margin=1.0, swap=False,  
reduction='mean')[source]#
```

Parameters

Shape:

Reference:

```
forward(anchor, positive, negative)[source]#
```

Returns:

Tensor

Code Examples

```

>>> # Initialize embeddings
>>> embedding = nn.Embedding(1000, 128)
>>> anchor_ids = torch.randint(0, 1000, (1,))
>>> positive_ids = torch.randint(0, 1000, (1,))
>>> negative_ids = torch.randint(0, 1000, (1,))
>>> anchor = embedding(anchor_ids)
>>> positive = embedding(positive_ids)
>>> negative = embedding(negative_ids)
>>>
>>> # Built-in Distance Function
>>> triplet_loss = \
>>>
nn.TripletMarginWithDistanceLoss(distance_function=nn.PairwiseDistance())
>>> output = triplet_loss(anchor, positive, negative)
>>> output.backward()
>>>
>>> # Custom Distance Function
>>> def l_infinity(x1, x2):
>>>     return torch.max(torch.abs(x1 - x2), dim=1).values
>>>
>>> triplet_loss = (
>>>
nn.TripletMarginWithDistanceLoss(distance_function=l_infinity,
margin=1.5))
>>> output = triplet_loss(anchor, positive, negative)
>>> output.backward()
>>>
>>> # Custom Distance Function (Lambda)
>>> triplet_loss = (
>>>     nn.TripletMarginWithDistanceLoss(
>>>         distance_function=lambda x, y: 1.0 -
F.cosine_similarity(x, y)))
>>> output = triplet_loss(anchor, positive, negative)
>>> output.backward()

```

Detailed Documentation

Documentation

Creates a criterion that measures the triplet loss given input tensors `aaa`, `ppp`, and `nnn` (representing anchor, positive, and negative examples, respectively), and a nonnegative, real-valued function (“distance function”) used to compute the relationship between the anchor and positive example (“positive distance”) and the anchor and negative example (“negative distance”).

The unreduced loss (i.e., with reduction set to 'none') can be described as:

where `NNN` is the batch size; `ddd` is a nonnegative, real-valued function quantifying the closeness of two tensors, referred to as the `distance_function`; and `marginmarginmargin` is a nonnegative margin representing the minimum difference between the positive and negative distances that is required for the loss to be 0. The input tensors have `NNN` elements each and can be of any shape that the distance function can handle.

If reduction is not 'none' (default 'mean'), then:

See also `TripletMarginLoss`, which computes the triplet loss for input tensors using the `lpl_plp` distance as the distance function.

`distance_function` (Callable, optional) – A nonnegative, real-valued function that quantifies the closeness of two tensors. If not specified, `nn.PairwiseDistance` will be used. Default: None

`margin` (float, optional) – A nonnegative margin representing the minimum difference between the positive and negative distances required for the loss to be 0. Larger

margins penalize cases where the negative examples are not distant enough from the anchors, relative to the positives. Default: 111.

swap (bool, optional) – Whether to use the distance swap described in the paper Learning shallow convolutional feature descriptors with triplet losses by V. Balntas, E. Riba et al. If True, and if the positive example is closer to the negative example than the anchor is, swaps the positive example and the anchor in the loss computation. Default: False.

reduction (str, optional) – Specifies the (optional) reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the sum of the output will be divided by the number of elements in the output, 'sum': the output will be summed. Default: 'mean'

Input: (N,*)(N, *)(N,*) where ** represents any number of additional dimensions as supported by the distance function.

Output: A Tensor of shape (N)(N)(N) if reduction is 'none', or a scalar otherwise.

V. Balntas, et al.: Learning shallow convolutional feature descriptors with triplet losses: <https://bmva-archive.org.uk/bmvc/2016/papers/paper119/index.html>

Runs the forward pass.